

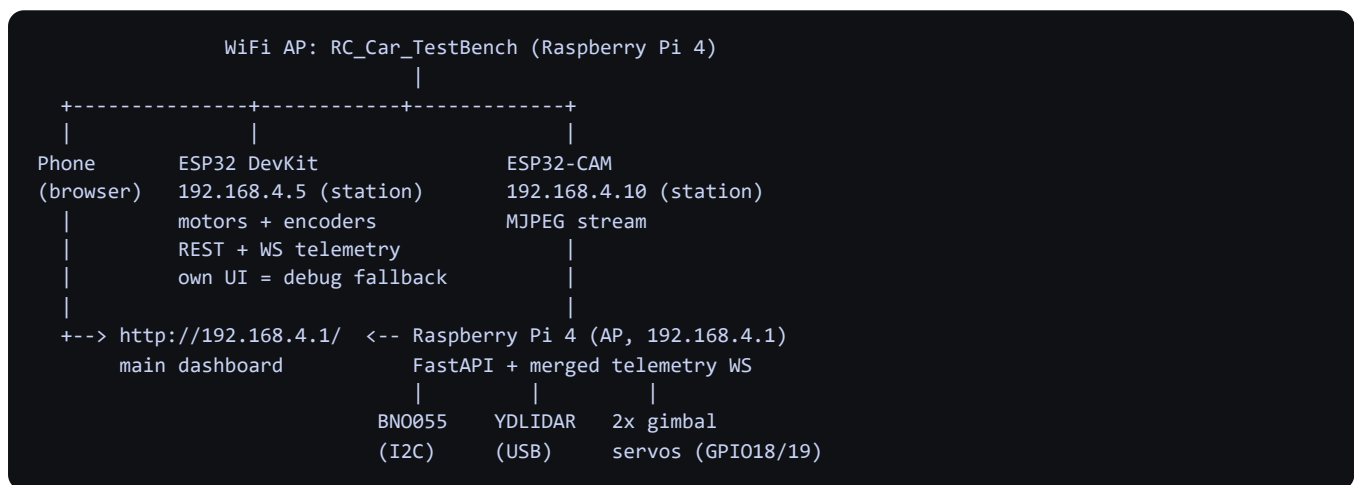
Hardware Architecture — Pi Centric

Status: current architecture. Supersedes *Hardware Architecture — Pi Integration Phase* (its wiring/power sections remain valid; its network topology does not).

The Raspberry Pi 4 is now the **brain** of the car: it hosts the WiFi network, serves the main dashboard, owns every sensor, and is where logging/autonomy grows. The ESP32 DevKit is reduced to what it is best at — hard-real-time motor control — and stays the safety authority.

System Topology

Three compute boards, one WiFi network — now hosted by the Pi.



- The **Pi** runs the AP and the main web dashboard (`pi/webapp` , port 80): e-stop, arm, hold-to-drive D-pad, per-wheel PWM/RPM, IMU, embedded CAM stream, gimbal sliders, LiDAR polar plot — one merged telemetry WebSocket at 10 Hz.
- The **DevKit** keeps its full REST/WS API (a stable contract) and its built-in web UI at `http://192.168.4.5` as a debug fallback. It remains the e-stop authority: STBY pull-downs cut the H-bridges in hardware.
- The **CAM** is unchanged — same SSID/PSK means the old firmware simply finds the “same” network. Browsers and the Pi pull MJPEG straight from it; video never passes through the DevKit and is never proxied through the Pi for plain viewing.

Network Plan

DEVICE	IP	HOW
Raspberry Pi 4 (AP + dashboard)	192.168.4.1	NetworkManager AP profile (<code>pi/setup_ap.sh</code>)
ESP32 DevKit	192.168.4.5	static, <code>STA_STATIC_IP</code> in <code>CarTestBench/config.h</code>
ESP32-CAM	192.168.4.10	static in <code>CamStreamer.ino</code> (unchanged)
Phone / laptop	DHCP 192.168.4.100–200	NM shared-mode dnsmasq

SSID `RC_Car_TestBench` , PSK `carbench123` — deliberately identical to the old ESP32-hosted AP so no station firmware changes.

AP Setup (on the Pi, one time)

```
cd ~/car/pi
bash setup_ap.sh
```

The script (NetworkManager, stock on Bookworm — no hostapd/dnsmasq packages to fight):

- Writes `dhcp-range=192.168.4.100,192.168.4.200,12h` to `/etc/NetworkManager/dnsmasq-shared.d/car-ap.conf` so the DHCP pool stays clear of the static boards.
- Turns off autoconnect on the old `car` station profile.
- Creates and raises the `car-ap` profile: wifi AP mode, band bg, WPA-PSK, `ipv4.method shared` , 192.168.4.1/24, autoconnect on boot.

While `car-ap` is active the Pi has no internet. For updates: `sudo nmcli c up <home-profile>` , then back with `sudo nmcli c up car-ap` .

Firmware Roles and Modes

BOARD	SKETCH	ROLE
ESP32 DevKit	<code>CarTestBench/</code>	motor controller: 4× TB6612 channels, 4 encoders, REST/WS API, fallback UI, e-stop latch, drive deadman
ESP32-CAM	<code>CamStreamer/</code>	MJPEG appliance at <code>:81/stream</code> — untouched by this migration
Raspberry Pi 4	<code>pi/</code>	AP, dashboard (<code>webapp/</code>), BNO055 (raw smbus2), YDLIDAR X2 (raw serial), gimbal, logging, future autonomy

`CarTestBench/config.h` → `WIFI_STATION_MODE` :

- **1 (current):** `WIFI_STA` , static 192.168.4.5, gateway .1, `WiFi.setAutoReconnect` plus a 5 s non-blocking reconnect watchdog in `loop()` (covers the boot-order case where the Pi’s AP rises after the DevKit).

- **0:** legacy self-hosted AP at 192.168.4.1 — the escape hatch. If the Pi is ever dead, flash with `0` and the old ESP32-centric rig is back.

Drive Deadman (safety)

`DRIVE_DEADMAN_MS = 500` in `config.h`. In the DevKit's `loop()` (non-blocking `millis()` check): if **any wheel has nonzero PWM** and no `/api/drive` or `/api/motor?...pwm=` command has arrived for 500 ms, all wheels **coast** and the event is logged to Serial. Wheels stay armed — the next command drives again.

Both UIs cooperate by re-sending the held command every 300 ms (Pi dashboard D-pad, ESP32 drive page D-pad, ESP32 motors-page sliders). Any Pi-side autonomy script must do the same (`DEADMAN_RESEND_S` in `pi/config.py`).

Layered safety, outermost first:

1. **STBY pull-downs** — bridges disabled in hardware until firmware arms.
2. **E-stop latch** — `POST /api/estop` from any UI or script; hardware STBY cut.
3. **Drive deadman** — dead client/link mid-drive stops the car in ≤ 0.5 s.
4. **Client deadman re-send** — a stalled control loop stops sending, which trips layer 3.

Control and Data Paths

FROM	TO	WHAT
Phone	Pi :80	dashboard page + merged telemetry WS (<code>/ws</code>)
Pi webapp	DevKit :80	control POSTs forwarded verbatim (<code>/api/drive</code> , <code>/api/motor</code> , <code>/api/estop...</code>)
Pi webapp	DevKit <code>/ws</code>	10 Hz wheel telemetry (background thread)
Pi webapp	CAM <code>/status</code>	1 Hz counters → FPS/bitrate in merged WS
Phone	CAM :81/stream	MJPEG, direct — never proxied
Pi (OpenCV, later)	CAM :81/stream	second direct stream for vision

The DevKit REST/WS API is unchanged and remains the contract; the Pi extends around it.

Boot Order

Pi first (brings up the AP), then DevKit and CAM in any order — both retry until the AP appears. In practice power everything at once: the Pi takes ~30 s to boot, the ESP32s just retry until it is up.

Migration / Flash Order

The order matters — flashing the DevKit to station mode before the Pi AP exists strands it (recoverable by re-flashing with `WIFI_STATION_MODE 0`):

- 1. Deploy `pi/` to the Pi, install deps, start `car-dashboard.service` .
- 2. Run `pi/setup_ap.sh` ; confirm the AP is up and the CAM reappears at 192.168.4.10.
- 3. Only then flash `CarTestBench` (station mode); confirm it at 192.168.4.5.
- 4. `CamStreamer` is **not** re-flashed.

What Changed vs the Pi Integration Phase

	PI INTEGRATION PHASE (SUPERSEDED)	PI CENTRIC (CURRENT)
AP	ESP32 DevKit	Raspberry Pi 4
Pi IP	192.168.4.20 (station)	192.168.4.1 (AP)
DevKit IP	192.168.4.1	192.168.4.5 (station)
Main UI	ESP32 <code>WebUI.h</code>	Pi <code>pi/webapp</code> (ESP32 UI = fallback)
Deadman	client-side convention only	enforced in firmware (500 ms)
CAM	192.168.4.10	192.168.4.10 (untouched)
LiDAR	RPLidar A1/C1 (planning placeholder)	YDLIDAR X2, confirmed on the bench
IMU driver	Adafruit CircuitPython BNO055 (planning placeholder)	raw smbus2 registers, no CircuitPython

Pi wiring unaffected by the LiDAR/IMU driver correction — BNO055 I2C address/clock-stretch workaround, gimbal servo power, general power architecture — still applies verbatim from the superseded doc. Its LiDAR section (RPLidar model/baud) does not; see `pi/README.md` and `pi/lidar.py` for the real YDLIDAR X2 wiring/protocol.